CENG-567
REINFORCEMENT LEARNING

Lecture-0

Course Introduction

Introduction to Reinforcement Learning

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Course Introduction-I

CENG 567, Reinforcement Learning (3 + 0)

2025/2026, Fall

Time: Thursday, 13:30-16:15

Classroom: D4, Computer Engineering Department

Instructor: Dr. Osman GÖKALP

Contact: osmangokalp@iyte.edu.tr

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Course Website

Teams: Lecture Notes, Assignments, Announcements

Teams enrollment code: ywxg6z8

The students are expected to **regularly check** the course's Teams class for announcements.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Course Textbooks & Resources

Main Textbook: Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020.

Supplementary Textbook: Reinforcement Learning, second edition: An Introduction, Richard S. Sutton and Andrew G. Barto, Bradford Books, second edition.

Other Resources: Selected papers, online materials, and instructor-provided notes may also be used throughout the course.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Course Outline (tentative)

1. 02 OCT 2025: Introduction to Reinforcement Learning (RL)
2. 09 OCT 2025 : Multi-Armed Bandits, Epsilon Greedy
3. 16 OCT 2025 : Upper Confidence Bounds, Thompson Sampling
4. 23 OCT 2025 : Contextual Bandits, Function Approximation
5. 30 OCT 2025 : Markov Chain, Markov Reward Process, Markov Decision Process
6. 06 NOV 2025: Dynamic Programming, Policy Iteration, Value Iteration
7. 13 NOV 2025 : Monte Carlo Methods
8. *20 NOV 2025 : Midterm Exam*
9. 27 NOV 2025 : Temporal Difference Learning, SARSA, Q-Learning
10. 04 DEC 2025: Deep Q-learning
11. 11 DEC 2025 : Actor-Critic Models, A2C
12. 18 DEC 2025 : Deep Deterministic Policiy Gradient
13. 25 DEC 2025 : Final Project Evaluation
14. *01 JAN 2026: No Lectures*
15. *Final Exam*

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Grading (tentative)

- Midterm Exam 30%
- Final Exam 40%

- * Project 30%
  - You are expected to develop an application that applies at least one reinforcement learning (RL) approach to solve a suitable problem.
  - The results should be presented in a paper following the IEEE format.

* You can work on a project in a group of **up to 3** people or individually.
Each group will **choose** its **own topic** based on their interests, subject to **instructor approval**.

Introduction to Reinforcement Learning

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Introduction to Reinforcement Learning

- **Reinforcement Learning** (**RL**) aims to create Artificial Intelligence (AI) **agents** that can make decisions in complex and uncertain environments.
  - with the goal of maximizing their long-term **benefit**
- Agents learn how to do this by interacting with their **environments**, which mimics the way we as humans learn from **experience.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The three paradigms of machine learning

1. Supervised Learning (SL)
   - *Given labeled data → learn input–output mapping*
   - Example: Predicting house prices from features (regression) / Classifying emails as spam or not spam (classification).
2. Unsupervised Learning (UL)
   - *Find hidden structure in unlabeled data*
   - Example: Customer segmentation in marketing / Clustering news articles by topic.
3. Reinforcement Learning (RL)
   - *Agent learns by interacting with an environment, receiving rewards/penalties*
   - Example: Training a robot to walk / Optimizing game-playing strategies.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Reinforcement Learning

- RL is a framework to learn how to make decisions **under uncertainty** to maximize a longterm benefit through **trial and error**.

- **Decisions** are made **sequentially**, and **earlier decisions affect** the situations and benefits that will be encountered later.
  - This separates RL from both SL and UL, which don't involve any decision-making.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Tasks

- Self-Driving Car

Using camera data (object positions, lane edges), the model learns how to steer and adjust speed to safely overtake other vehicles while maintaining efficiency.

- Inventory Management

From historical sales and supplier lead times, the model learns when and how much to reorder, ensuring seasonal demand is met while minimizing storage and transportation costs.

How RL Differs from SL & UL?

- **Output**: RL gives *decisions* (actions), not predictions or clusters.
- **No supervisor**: Learns from *experience & feedback* instead of ground-truth labels.
- **Trial and error**: Example → speeding too much may cause accidents; over-ordering inventory leads to waste.
- **Uses SL/UL outputs**: e.g., image recognition (SL) → input for driving decisions; forecasting (SL) → input for inventory control.
- **Predictive element**: Considers how current actions shape *future states*.
- **Shared methods**: Neural networks and other ML tools are also used in RL.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

RL as a Decision-Making Framework

- RL is fundamentally about **decision-making**.

- Inspired by **human learning**: learning from experience, not blueprints.

- **Analogy – Toddler with toy blocks**:
  - Taller tower = success (reward).
  - Collapse = failure (penalty).
  - Learns stability through repeated attempts.
  - Builds better strategies based on past outcomes.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why RL?

- RL mirrors **human trial-and-error learning**.

- Identifies actions with the highest **long-term rewards**.

- Proven success stories in:
  - Game playing (e.g., AlphaGo).
  - Robotics & automation.
  - Resource management.
  - Self-driving cars.

- RL is a transformative AI paradigm shaping future industries.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Examples: TD-Gammon (1992, IBM Research – Gerald Tesauro)

- **First famous Reinforcement Learning (RL) implementation**
- Learned to play **superhuman-level backgammon**
- Handled **1,020 possible configurations** of the game
- **Impact:**
  - Generated excitement in the backgammon community
  - Introduced novel strategies that even **taught humans**
- **Core methods (still central today):**
  - Temporal-Difference (TD) learning (TD-lambda)
  - Self-play (agent learns by **playing against itself** instead of relying on human opponents or pre-collected data)
  - Use of neural networks

This Photo by Unknown Author is licensed under CC BY

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Super-Human Performance in Atari Games (2015, DeepMind – Volodymyr Mnih et al.)

- **Seminal RL breakthrough**
- RL agents trained to play **Atari 2600 games**
- Input: **raw screen pixels + game score**
- No hand-crafted or game-specific features
- Powered by **Deep Neural Networks**
- Algorithm: **Deep Q-Network (DQN)**
    - Achieved **super-human performance** in many games
    - Became one of the **most popular RL algorithms**

This Photo by Unknown Author is licensed under CC BY-SA-NC

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Beating World Champions in Go, Chess, and Shogi (DeepMind)

- **AlphaGo (2015–2016)**
  - First to beat a professional Go player (2015)
  - Defeated world champion **Lee Sedol** (2016)
  - Inspired the documentary *AlphaGo*
  - Training: **human expert moves + RL self-play**
- **AlphaGo Zero (2017)**
  - Trained **only through self-play** (no human data)
  - Defeated original AlphaGo **100–0**
- **AlphaZero (2018)**
  - Generalized to **Go, Chess, Shogi**
  - Learned **only from game rules**, no human input
  - Achieved **superhuman play within hours** on TPUs
  - Praised for **unconventional strategies** by Garry Kasparov (chess) & Yoshiharu Habu (shogi)

This Photo by Unknown Author is licensed under CC BY-SA

Example: Victories in Complex Strategy Games

- RL expanded beyond Atari & board games → into:
  - **Mario, Quake III Arena, Capture the Flag**
  - **Dota 2, StarCraft II**
- **Key Challenges:**
  - Strategic long-term planning
  - Multi-agent decision-making (game theory)
  - Imperfect information
  - Huge action & state spaces
- **Massive Training Resources:**
  - **OpenAI Five (Dota 2, 2018)**
    - Trained with **256 GPUs + 128,000 CPU cores**
    - ≈ **900 years of game experience/day**
  - **AlphaStar (StarCraft II, DeepMind, 2019)**
    - Hundreds of models trained
    - Each with ≈ **200 years of game experience**
    - Initially trained on **human game data**

This Photo by Unknown Author is licensed under CC BY

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Elevator Optimization with RL (1996)

- **Researchers:** Robert Crites & Andrew Barto
- **Problem:** Optimize elevator dispatching in a **10-story building with 4 elevators**
- **Challenges:**
  - Far larger **state space** than TD-Gammon
  - **Partial observability** (e.g., number of people waiting was hidden)
  - Many possible **decision options** at each step
- **Results:**
  - RL model outperformed the best heuristics of the time
  - Significant improvements in:
    - **Average passenger wait time**
    - **Passenger travel time**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Humanoid Robots & Dexterous Manipulation

- **2017 – DeepMind (Nicolas Heess et al.)**
  - Trained simulated bodies (humanoids, others) for **locomotion behaviors**: running, jumping, etc.
- **2018 – OpenAI (Marcin Andrychowicz et al.)**
  - Trained a **five-fingered humanoid hand** to manipulate a block from start → goal
- **2019 – OpenAI (Ilge Akkaya et al.)**
  - Trained a **robot hand to solve a Rubik's cube**
- **Key Techniques:**
  - Training **in simulation**
  - Successful **transfer to real robots** using **domain randomization**
- **Impact:** Demonstrated RL can solve **complex, high-dimensional, real-world robotics tasks**

(a) Simulation (b) Physical robot

Figure 1.1 – OpenAI's RL model that solved a Rubik's cube is trained in simulation (a) and deployed on a physical robot (b) (image source: OpenAI Blog, 2019)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: RL in Real-World Autonomous Systems

This Photo by Unknown Author is licensed under CC BY-SA-NC

Emergency Response Robots

- Operate in **dangerous or disaster environments**
- Tasks include:
  - Locating survivors in damaged structures
  - Turning off gas valves, manipulating debris
- **Benefit:** Scale emergency operations beyond human capabilities

Self-Driving Vehicles & Drones

- **Modern autonomous cars** exist (Waymo, Cruise, Tesla)
- RL is used to **train agents for specific tasks**, e.g.:
  - Cars: Self-parking, lane-changing, obstacle avoidance
  - Drones: Takeoff, landing, collision avoidance
- **Goal:** Enable safer, adaptive autonomous systems

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Reinforcement Learning in Supply Chain Management

- Many supply chain decisions are **sequential and uncertain**, making RL a natural fit

Applications:

- **Inventory Planning:**
  - Decide **when and how much to order**
  - Minimize **shortages, excess inventory, spoilage, and costs**
- **Bin Packing:**
  - Efficiently place items into containers
  - Minimize the **number of containers** and ensure smooth operations
  - **Complex combinatorial problem** suitable for RL

- **Benefit:** RL helps **optimize operations** and **reduce costs** in complex, dynamic supply chains

This Photo by Unknown Author is licensed under CC BY-SA-NC

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Reinforcement Learning in Manufacturing

- **Impact:** Automate manual tasks → **reduce costs** & **increase quality**
- **Companies:** Many are exploring RL integration in production environments

Applications:

- **Machine Calibration:**
  - Replace human expert calibration → more **efficient and accurate**
- **Chemical Plant Operations:**
  - Sequential decision-making → **better product quality**, less equipment wear
- **Equipment Maintenance:**
  - Plan downtimes → balance **cost of downtime vs breakdown risk**
- **Robotics Transfer:**
  - RL solutions from robotics can be adapted to manufacturing tasks

- **Benefit:** RL enables **smart, autonomous, and optimized manufacturing systems**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Reinforcement Learning in Personalization & Recommender Systems

- **Impact:** RL has created **significant business value** in personalization for big tech companies

Applications:

- **Advertising:**
  - Optimize **order & content** of promotions → higher **conversion & satisfaction**
- **News Recommendation:**
  - Example: **Microsoft News** improved visitor engagement by sequencing articles
- **Entertainment:**
  - **Netflix** personalizes artwork and title recommendations → viewers find **relevant content faster**
- **Healthcare:**
  - RL assists in **choosing effective treatments** → better outcomes at reduced costs

- **Benefit:** RL enables **dynamic, user-tailored decisions** across industries

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: Reinforcement Learning in Smart Cities

- **Traffic Management:**
  - Multi-intersection traffic lights coordinated using **multi-agent RL**
  - Ensures **smooth traffic flow** and reduces congestion
- **Electricity Grid Management:**
  - Balance **generation and demand** in real time
  - Control **EV charging, AC systems**, etc., without sacrificing service quality
  - RL helps maintain **grid safety and efficiency**
- **Takeaway:** RL has **huge potential** across industries

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Introduction to RL Concepts

- **Agent:** The learner at the center of RL
  - Single agent → standard RL
  - Multiple agents → Multi-Agent RL (MARL)
    - Interactions: **cooperative, competitive, or mixed**
- **Environment:** Everything **outside the agent**
- **State (s):** Complete information describing the environment
  - Fully observable → agent sees the entire state
  - Partially observable → agent sees **only part** (or derivation) of the state

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Observations vs States

- **Observation:** What the agent actually receives from the environment
  - Sometimes equal to the state
  - Sometimes partial or derived from the state
- **Note:** Terms “state” and “observation” are sometimes used interchangeably

Actions and Rewards

- **Action:** Decision the agent makes in a given state/observation

- **Reward:** Scalar signal from the environment indicating how good the action was
    - Could be positive, negative, or zero

- **Goal:** Maximize cumulative reward over time

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

RL Process Diagram

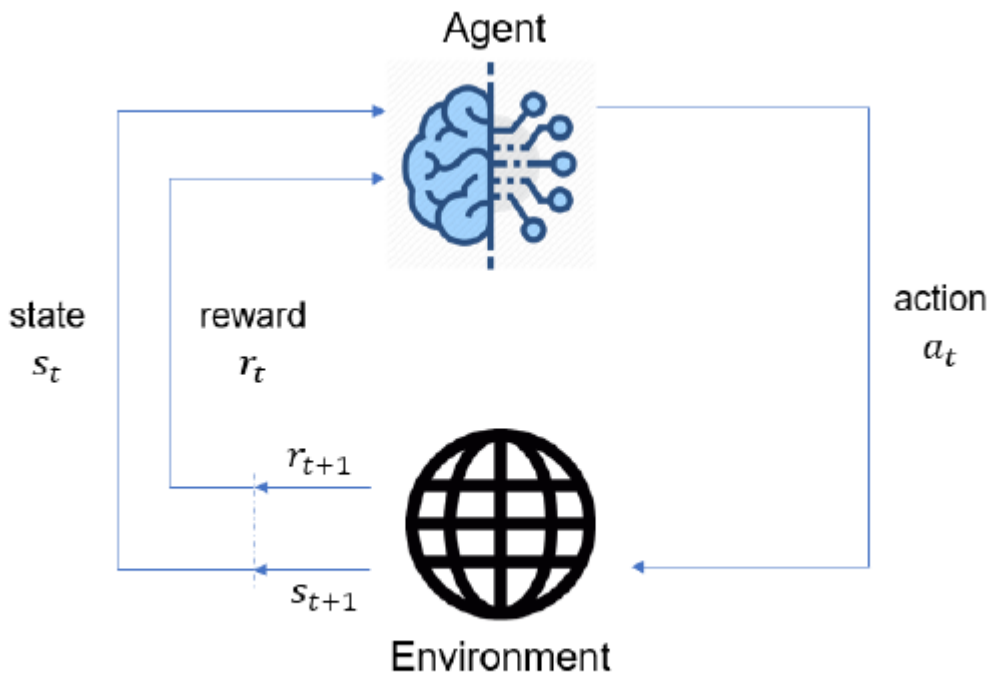

Figure 1.2 – RL process diagram

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Long-Term Goals

- Some actions yield **immediate reward but poor long-term outcome**
- Some actions yield **low immediate reward but high long-term payoff**
- **Objective:** Maximize **cumulative reward** over the relevant horizon

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Episodic vs Continuing Tasks

Episodic Task: Finite horizon, sequence from initial → terminal state
•Goal: Maximize **total cumulative reward** in an episode

Continuing Task: Infinite horizon
•Goal: Maximize **average reward** over time

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Policy

- **Policy (π):** Maps observations → actions
- **Objective:** Find the policy that **maximizes expected cumulative reward**
- Core idea: RL is all about **learning the best actions over time**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Tic-Tac-Toe as an RL Problem

- **Game:** Two players take turns marking empty spaces on a **3 × 3 grid**
- **Goal:** Place **three marks in a row** (vertical, horizontal, diagonal)
- **Draw:** All spaces filled with no winner
- **RL Mapping:**
  - **Agent:** The player learning to play
  - **Environment:** The board + opponent moves
  - **State:** Current board configuration
  - **Action:** Place a mark in an empty space
  - **Reward:** +1 win, 0 draw, -1 loss

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Board State Example

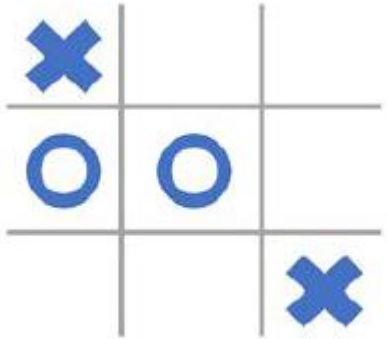

Figure 1.3 – An example board configuration in tic-tac-toe

- **Board = Environment**, fully observable to the agent
- **Number of unique states:** 765 (excluding rotations and reflections)
- **Objective:** Learn a **policy** that maps each state → best action

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Episodic Task & Rewards

- **Episodic RL Task:**
- Game lasts a maximum of 9 turns → terminal state reached
- Terminal state:
    - 3 Xs or Os in a row → win
    - All spaces filled → draw

- **Rewards:**
- +1 for win
- -1 for loss
- 0 for draw
- **Intermediate moves:** reward = 0

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Summary

- **What we covered:**
  - Definition and **importance of RL** as the next frontier in AI
  - **Success stories & real-world applications** (games, robotics, industry, smart cities)
  - **Fundamental RL concepts:** agent, environment, state, action, reward, policy
- **Purpose of this chapter:**
  - Refresh your understanding of RL
  - Motivate and prepare for **advanced RL topics**
- **Next up:**
  - **Multi-Armed Bandit problems** – a key RL algorithm class with applications in **personalization and advertising**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."